Python(22AIE205) Programming Lab

Weekly Lab Report-6

LAB 6: Functions

Name:	C.Pranaveswar
Roll No.	AIE25102
Section:	AIE-B
Department:	Artificial Intelligence and Engineering
Experiment Date:	25-08-2026

1. Experiment Record

Date: 25-08-2026

Experiment Title: Functions

Aim: To practise defining and using Functions in Python using Python IDLE.

Software Required: Python 3.x and Python IDLE

2. Questions Given

The following questions were assigned for Lab 6. The detailed programs and sample outputs are presented in the next section.

Question 1: Write a function to return the square and cube of a given number.

Question 2: Write a function to check whether a given number is even or odd.

Question 3: Write a function to calculate the factorial of a number, and print the factorials from 1 to n.

Question 4: Write a function to check whether a number is prime, and print all prime numbers up to n.

Question 5: Write a function to find the sum of the digits of a number.

Question 6: Write a function to reverse the digits of a given number.

Question 7: Write a function to check whether a given number is a palindrome.

Question 8: Write a function to check whether a given number is an Armstrong number.

Question 9: Write a function to print the first n terms of the Fibonacci series.

Question 10: Write functions to find the GCD and LCM of two numbers.

Question 11: Write a function to find the largest of three numbers.

Question 12: Write a function to check whether a given year is a leap year.

Question 13: Write a function to swap two numbers.

Question 14: Write functions to calculate simple interest and compound interest.

Question 15: Write a function with a default argument to print the multiplication table of a number.

Question 16: Write a function to count the number of digits in a given number.

Question 17: Write a function to check whether a given number is a perfect number.

Question 18:Write a function is_perfect(n) that returns True if a number is a perfect number

Question 19: Write a function reverse_string(text) that returns the reverse of a string without using any built-in reverse method.

Question 20:Write a function to check whether a given string is a palindrome, using the reverse function written above.

Question 21: Write a function count_vowels_consonants(text) that returns the count of vowels and the count of consonants as two values.

Question 22:Write a function count_words(sentence) that returns the number of words in a sentence.

Question 23:Write a function that accepts a string and returns it in upper case without using the built-in upper() method

Question 24:Write a function count_character(text, ch) that returns the number of times a given character occurs in a string.

Question 25: Write a function find_largest(numbers) that returns the largest element of a list without using the built-in max() function.

Question 26:Write a function find_average(numbers) that returns the average of the elements of a list.

Question 27: Write a function separate_even_odd(numbers) that returns two lists, one containing the even numbers and the other the odd numbers.

Question 28: Write a function search_element(numbers, key) that returns the position of an element in a list, and returns −1 if it is not present.

Question 29: Write a function count_positive_negative(numbers) that returns the count of positive numbers and the count of `negative numbers.`

Question 30:Write a function sort_list(numbers) that returns a list sorted in ascending order without using the built-in sort() method.

Question 31:Write a function remove_duplicates(numbers) that returns a list after removing the repeated elements.

Question 32:Write a function total_marks(*subjects) that accepts any number of marks and returns their total.

Question 33:Write a function merge_lists(list1, list2) that returns a single list containing the elements of both the lists.

Question 34:Write a function that accepts a list of numbers and returns the sum of only the elements present at the even positions.

Question 35:Student Result. Read the roll number, name and the marks in three subjects. Write the functions calculate_total(), calculate_average(), find_grade() and find_result(), and display all the details. Grade: A for an average of 90 and above, B for

75 and above, C for 60 and above, D for 40 and above and F below 40. The result is PASS only if the marks in every subject are

40 or above.

Question 36:ATM Simulation. Write a menu-driven program that verifies a PIN, allowing three attempts, and then permits the customer to check the balance, withdraw cash and deposit cash. Write the functions check_pin(), is_valid_amount(), withdraw()

and deposit(), where the last two return the updated balance.

Question 37: Simple Calculator. Write separate functions for addition, subtraction, multiplication and division, and a menu-driven main program that calls the appropriate function. Division by zero must be handled inside the function.

Question 38:Electricity Bill. Write a function that returns the bill amount for the units consumed, using the slab rates Rs. 3 per unit for the first 100 units, Rs. 5 per unit for the next 200 units and Rs. 8 per unit thereafter.

Question 39:Employee Salary. Write functions to compute the dearness allowance at 40% of the basic pay, the house rent allowance at 20% of the basic pay, and the net salary. Display the complete salary slip.

Question 40:Library Management. Write a menu-driven program with the functions add_book(), display_books(), search_book() and delete_book(), where the search function returns the position of the book and returns −1 when the book is

absent.

Question 41: Number Analysis Menu. Write a menu-driven program offering the options factorial, prime check, sum of digits, palindrome check and Armstrong check, by reusing the functions written in Section A. The menu must repeat until the user

chooses to exit.

3. Programs, Sample Outputs and Results

Question 1: Write a function to return the square and cube of a given number.

Program:

```
def square_cube(n):
    square = n * n
    cube = n * n * n
    return square, cube
    n = int(input("Enter number: "))
```

square, cube = square_cube(n)

```
print(square)
print(cube)
```

Input and Output:

```
>>>
===== RESTART: C:/Users/prana/OneD
Enter number: 23
529
12167
>>>
```

Result: The program correctly used a function returning multiple values to compute the square and cube of 23, correctly returning 529 and 12167.

Question 2: Write a function to check whether a given number is even or odd.

Program:

```
def is_even(n):
    if n % 2 == 0:
        return True
    else:
        return False
```

```
        n = int(input())
        print(is_even(n))
```

Input and Output:

```
>>
===== RESTART: C:/Users/prana/One
4
True
>>
```

Result: The program correctly used the modulus operator to check divisibility by 2

Question 3: Write a function to calculate the factorial of a number, and print the factorials from 1 to n.

Program:

```
def factorial(n):
    fact = 1
    for i in range(1, n + 1):
        fact = fact * i
        return fact
        n = int(input())
        for i in range(1,n+1):
            print(factorial(i))
```

Input and Output:

```
>>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\3.py ===
4
1
2
6
24
>>>
```

Result: The program correctly used an iterative function to compute the factorial of each value from 1 to 4, correctly returning

1, 2, 6, 24

Question 4: Write a function to check whether a number is prime, and print all prime numbers up to n.

Program:

```
def is_prime(n):
    if n < 2:
        return False
        for i in range(2, n):
            if n % i == 0:
                return False
                return True
                n = int(input())
                for i in range(1,n+1):
                    if is_prime(i):
                        print(i)
```

Input and Output:

```
>>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE2510
10
2
3
5
7
>>>
```

Result: The program correctly used a helper function to test primality of every number up to 10, correctly listing 2, 3,5,7

Question 5: Write a function to find the sum of the digits of a number.

Program:

```
def sum_of_digits(n):
    total = 0
    while n > 0:
        digit = n % 10
        total = total + digit
        n = n // 10
        return total
        n = int(input("Enter number: "))
        print(sum_of_digits(n))
```

Input and Output:

```
>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\5.py =====
Enter number: 40
4
>
```

Result: The program correctly extracted each digit using modulus and floor division and accumulated the total, correctly returning 4 for the input 40.

Question 6: Write a function to reverse the digits of a given number.

Program:

```
def reverse_number(n):
    rev = 0
    while n > 0:
        digit = n % 10
        rev = rev * 10 + digit
        n = n // 10
        return rev
        n = int(input("Enter number: "))
        print(reverse_number(n))
```

Input and Output:

```
>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\6.py =
Enter number: 546
645
>>
```

Result:

The program correctly rebuilt the number digit by digit in reverse order, correctly returning 654 for the input 546.

Question 7: Write a function to check whether a given number is a palindrome.

Program:

```
def reverse_number(n):
    rev = 0
    while n > 0:
        digit = n % 10
        rev = rev * 10 + digit
        n = n // 10
        return rev
        def is_palindrome(n):
            if n == reverse_number(n):
                return True
            else:
                return False
                n = int(input())
                print(is_palindrome(n))
```

Input and Output:

```
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\7.py
23432
True
```

Result: The program correctly reused the reverse_number function to compare a number with its reverse, correctly returning True for the palindrome 23432.

Question 8: Write a function to check whether a given number is an Armstrong number.

Program:

```
def is_armstrong(n):
    o = n
    total = 0
    digits = len(str(n))
    while n > 0:
        digit = n % 10
        total = total + digit ** digits
        n = n // 10
        if total == o:
            return True
        else:
            return False
            n = int(input("Enter number: "))
            print(is_armstrong(n))
```

Input and Output:

```
>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\8.py ==
Enter number: 12321
False
>>
```

Result: The program correctly raised each digit to the power of the digit count and summed the results, correctly returning false for the Armstrong number 12321.

Question 9: Write a function to print the first n terms of the Fibonacci series.

Program:

```
def fibonacci(n):
    a = 0
    b = 1
for i in range(n):
    print(a, end=" ")
    c = a + b
    a = b
    b = c
    n = int(input())
    fibonacci(n)
```

Input and Output:

```
>>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\9.py ===
10
0 1 1 2 3 5 8 13 21 34
```

Result: The program correctly generated the Fibonacci sequence iteratively, correctly printing the first 10 terms as 0 1 1 2 3 5 8

13 21 34.

Question 10: Write functions to find the GCD and LCM of two numbers.

Program:

```
def find_gcd(a, b):
    while b != 0:
```

a, b = b, a % b

```
    return a
def find_lcm(a, b):
    return (a * b) // find_gcd(a, b)
    a = int(input())
    b = int(input())
    print(find_gcd(a, b))
    print(find_lcm(a, b))
```

Input and Output:

```
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\10.py =
23
42
1
966
```

Result:

The program correctly used the Euclidean algorithm to compute the GCD and derived the LCM from it, correctly

Question 11: Write a function to find the largest of three numbers.

Program:

```
def largest_of_three(a, b, c):
    if a > b and a > c:
        return a
```

```
    elif b > c:
        return b
    else:
        return c
        a = int(input())
        b = int(input())
        c = int(input())
        print(largest_of_three(a, b, c))
```

Input and Output:

```
>>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\11.py ====
23
12
11
23
>>>
```

Result: The program correctly compared all three values using nested conditions, correctly returning 23

Question 12: Write a function to check whether a given year is a leap year.

Program:

```
def is_leap_year(year):
    if year % 400 == 0:
        return True
    elif year % 100 == 0:
        return False
    elif year % 4 == 0:
        return True
    else:
        return False
        year = int(input())
        print(is_leap_year(year))
        print(is_leap_year(year))
        print(is_leap_year(year))
```

Input and Output:

```
>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\12.py
2019
False
```

Result:

The program correctly applied the century and 4-year divisibility rules, correctly returning false

Question 13: Write a function to swap two numbers.

Program:

```
def swap(a, b):
    return b, a
    a = int(input())
    b = int(input())
```

a, b = swap(a, b)

```
print(a)
print(b)
```

Input and Output:

```
>>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\13.py
23
21
21
23
```

Result:

The program correctly used tuple unpacking inside the function to swap the two values, correctly returned

Question 14: Write functions to calculate simple interest and compound interest.

Program:

```
def simple_interest(p, r, t):
    return (p * r * t) / 100
    def compound_interest(p, r, t):
        amount = p * (1 + r / 100) ** t
        return amount - p
        p = float(input())
        r = float(input())
        t = float(input())
        print(simple_interest(p, r, t))
        print(compound_interest(p, r, t))
```

Input and Output:

```
>>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\14.py ====
10
10
10
10.0
15.937424601000025
>>>
```

Result: The program correctly applied the simple interest and compound interest formulas

Question 15: Write a function with a default argument to print the multiplication table of a number.

Program:

```
def print_table(n, limit=10):
    for i in range(1, limit + 1):
        print(n, "x", i, "=", n * i)
        n = int(input("Enter number: "))
        print_table(n)
```

Input and Output:

```
>>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\15.py
Enter number: 23
23 x 1 = 23
23 x 2 = 46
23 x 3 = 69
23 x 4 = 92
23 x 5 = 115
23 x 6 = 138
23 x 7 = 161
23 x 8 = 184
23 x 9 = 207
23 x 10 = 230
>>>
```

Result:

The program correctly used a default argument of 10 for the limit parameter

Question 16: Write a function to count the number of digits in a given number.

Program:

```
def count_digits(n):
    count = 0
    while n > 0:
        count = count + 1
        n = n // 10
        return count
        n = int(input())
        print(count_digits(n))
```

Input and Output:

```
>>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\16.py =
12
2
>>>
```

Result: The program correctly counted digits by repeatedly dividing the number by 10

Question 17: Write a function to check whether a given number is a perfect number.

Program:

```
def is_perfect(n):
    total = 0
    for i in range(1, n):
        if n % i == 0:
            total = total + i

if total == n:
    return True
else:
    return False
    n = int(input())
    print(is_perfect(n))
```

Input and Output:

```
>>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\17.py ====
23
False
>>>
```

Result: The program correctly summed all proper divisors of the number and compared the sum to the original number, correctly returning false for the perfect number 23.

Question 18:Write a function is_perfect(n) that returns True if a number is a perfect number

Program:

```
def is_perfect(n):
    total = 0
    for i in range(1, n):
        if n % i == 0:
            total = total + i
            if total == n:
                return True
            else:
                return False
                n = int(input())
                print(is_perfect(n))
```

Input and Output:

```
>>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\18.py
23
False
>>>
```

Result: The function correctly identified whether a number is perfect by iterating through all potential proper divisors, summing

those that divide the number evenly, and comparing the total to the original input, returning True for the number

Question 19:Write a function reverse_string(text) that returns the reverse of a string without using any built-in reverse

method.

Program:

```
def reverse_string(text):
    rev = ""
    for ch in text:
        rev = ch + rev
        return rev
        text = input()
        print(reverse_string(text))
```

Input and Output:

```
>>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\19.py ==
pranav
vanarp
>>>
```

Result:The function successfully reversed the string by iterating through each character

Question 20:Write a function to check whether a given string is a palindrome, using the reverse function written above.

Program:

```
def reverse_string(text):
    rev = ""
    for ch in text:
        rev = ch + rev
        return rev
        def is_palindrome(text):
            if text == reverse_string(text):
                return True
            else:
                return False
                text = input()
                print(is_palindrome(text))
```

Input and Output:

```
>>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\20.py
madam
True
>>>
```

Result:The function correctly checked for symmetry by comparing the original string to the result of reverse_string(), returning True

for the input "madam"

Question 21:Write a function count_vowels_consonants(text) that returns the count of vowels and the count of consonants as two values.

Program:

```
def count_vowels_consonants(text):
    vowels = 0
    consonants = 0
    for ch in text:
        if ch in "aeiouAEIOU":
            vowels = vowels + 1
        elif ch.isalpha():
            consonants = consonants + 1
            return vowels, consonants
            text = input("Enter string: ")
```

v, c = count_vowels_consonants(text)

```
print(v)
print(c)
```

Input and Output:

```
>>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\21.py ====
Enter string: pranav
2
4
```

Result:The program accurately categorized characters into vowels and alphabetic consonants

Question 22:Write a function count_words(sentence) that returns the number of words in a sentence.

Program:

```
def count_words(sentence):
    words = sentence.split()
    return len(words)
    sentence = input()
    print(count_words(sentence))
```

Input and Output:

```
>>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\22.py
worD
1
```

Result:The program correctly calculated word count by splitting the input string on whitespace

Question 23: Write a function that accepts a string and returns it in upper case without using the built-in upper()

method.

Program:

```
def to_upper(text):
    result = ""
    for ch in text:
        if 'a' <= ch <= 'z':
            result = result + chr(ord(ch) - 32)
        else:
            result = result + ch
            return result
            text = input()
            print(to_upper(text))
```

Input and Output:

```
>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\23.py ==
Pranv
PRANV
>>
```

Result:The function successfully converted lowercase characters to uppercase using ASCII value offset subtraction

Question 24: Write a function count_character(text, ch) that returns the number of times a given character occurs in a

string.

Program:

```
def count_character(text, ch):
    count = 0
    for x in text:
        if x == ch:
            count = count + 1
            return count
            text = input()
```

```
    ch = input()
    print(count_character(text, ch))
```

Input and Output:

```
>>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\24.py ==
pranav
1
0
>>>
```

Result:The program accurately tracked character occurrences by iterating through the string and matching against the target character

Question 25: Write a function find_largest(numbers) that returns the largest element of a list without using the built-in

max() function.

Program:

```
def find_largest(numbers):
    largest = numbers[0]
    for n in numbers:
        if n > largest:
            largest = n
            return largest
            numbers = list(map(int, input("Enter numbers: ").split()))
print(find_largest(numbers))
```

Input and Output:

```
>>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\25.py =
Enter numbers: 23 43
43
>>>
```

Result:The function correctly identified the maximum value by iteratively comparing each element against the current maximum

Question 26: Write a function find_average(numbers) that returns the average of the elements of a list.

Program:

```
def find_average(numbers):
    total = 0
    for n in numbers:
        total = total + n
        return total / len(numbers)
        numbers = list(map(int, input("Enter numbers: ").split()))
        print("Average =", find_average(numbers))
```

Input and Output:

```
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\26.py ==
Enter numbers: 23 31 33
Average = 29.0
```

Result:

The program correctly uses find_average() to calculate the average of all elements in the list.

Question 27: Write a function separate_even_odd(numbers) that returns two lists, one containing the even numbers and the

other the odd numbers.

Program:

```
def separate_even_odd(numbers):
    even = []
    odd = []
    for n in numbers:
        if n % 2 == 0:
            even.append(n)
        else:
            odd.append(n)
            return even, odd
            numbers = list(map(int, input("Enter numbers: ").split()))
            even, odd = separate_even_odd(numbers)
            print("Even numbers =", even)
            print("Odd numbers =", odd)
```

Input and Output:

```
>>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\27.py =
Enter numbers: 32 31
Even numbers = [32]
Odd numbers = [31]
>>>
```

Result:

The program correctly separates the input list into even and odd numbers.

Question 28: Write a function search_element(numbers, key) that returns the position of an element in a list, and returns −1

if it is not present.

Program:

```
def search_element(numbers, key):
    for i in range(len(numbers)):
        if numbers[i] == key:
            return i
            return -1
            numbers = list(map(int, input("Enter numbers: ").split()))
            key = int(input("Enter element to search: "))
            print("Position =", search_element(numbers, key))
```

Input and Output:

```
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\28.py ===
Enter numbers: 32 21 33
Enter element to search: 33
Position = 2
```

Result:

The program correctly returns the zero-based position of the first matching element and −1 when it is absent.

Question 29: Write a function count_positive_negative(numbers) that returns the count of positive numbers and the count

of negative numbers.

Program:

```
def count_positive_negative(numbers):
    positive = 0
    negative = 0
    for n in numbers:
        if n > 0:
            positive += 1
        elif n < 0:
            negative += 1
            return positive, negative
            numbers = list(map(int, input("Enter numbers: ").split()))
```

p, n = count_positive_negative(numbers)

```
print("Positive numbers =", p)
print("Negative numbers =", n)
```

Input and Output:

```
>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\29.py ===
Enter numbers: 21 22 31
Positive numbers = 3
Negative numbers = 0
>>
```

Result:

The program correctly counts positive and negative numbers. Zero is not counted in either category.

Question 30: Write a function sort_list(numbers) that returns a list sorted in ascending order without using the built-in

sort() method.

Program:

```
def sort_list(numbers):
    result = numbers.copy()
    for i in range(len(result)):
        for j in range(0, len(result) - i - 1):
            if result[j] > result[j + 1]:
                result[j], result[j + 1] = result[j + 1], result[j]
                return result
                numbers = list(map(int, input("Enter numbers: ").split()))
                print("Sorted list =", sort_list(numbers))
```

Input and Output:

```
>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\30.py ==
Enter numbers: 32 21 12
Sorted list = [12, 21, 32]
>>
```

Result:

The program sorts the list in ascending order without using the built-in sort() method.

Question 31: Write a function remove_duplicates(numbers) that returns a list after removing the repeated elements.

Program:

```
def remove_duplicates(numbers):
    result = []
    for n in numbers:
        if n not in result:
            result.append(n)
            return result
            numbers = list(map(int, input("Enter numbers: ").split()))
            print("List without duplicates =", remove_duplicates(numbers))
```

Input and Output:

```
>>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\31.py
Enter numbers: 23 12 21 32
List without duplicates = [23, 12, 21, 32]
>>>
```

Result:

The program removes repeated elements while preserving the first occurrence of each value.

Question 32: Write a function total_marks(*subjects) that accepts any number of marks and returns their total.

Program:

```
def total_marks(*subjects):
    total = 0
    for mark in subjects:
        total += mark
        return total
        marks = list(map(int, input("Enter marks: ").split()))
        print("Total marks =", total_marks(*marks))
```

Input and Output:

```
List without duplicates = [23, 12, 21, 32]
>>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\32.py ==
Enter marks: 39 81 43
Total marks = 163
>>>
```

Result:

The program correctly accepts any number of marks using *subjects and returns their total.

Question 33: Write a function merge_lists(list1, list2) that returns a single list containing the elements of both the lists.

Program:

```
def merge_lists(list1, list2):
    return list1 + list2
    list1 = list(map(int, input("Enter first list: ").split()))
    list2 = list(map(int, input("Enter second list: ").split()))
    print("Merged list =", merge_lists(list1, list2))
```

Input and Output:

```
>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\33.py =
Enter first list: 1 2 3
Enter second list: 4 6 7
Merged list = [1, 2, 3, 4, 6, 7]
>>
```

Result:

The program correctly combines both input lists into a single list.

Question 34: Write a function that accepts a list of numbers and returns the sum of only the elements present at the even

positions.

Program:

```
def sum_even_positions(numbers):
    total = 0
    for i in range(0, len(numbers), 2):
        total += numbers[i]
        return total
        numbers = list(map(int, input("Enter numbers: ").split()))
        print("Sum at even positions =", sum_even_positions(numbers))
```

Input and Output:

```
>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\34.py ==
Enter numbers: 2 12 21 3
Sum at even positions = 23
>>
```

Result:

The program adds the elements at zero-based even positions

Question 35: Student Result. Read the roll number, name and the marks in three subjects. Write the functions

calculate_total(), calculate_average(), find_grade() and find_result(), and display all the details. Grade: A for an average of 90 and above, B for 75 and above, C for 60 and above, D for 40 and above and F below 40. The result is PASS only if the marks in every subject are 40 or above.

Program:

```
def calculate_total(m1, m2, m3):
    return m1 + m2 + m3
    def calculate_average(total):
```

```
            return total / 3
            def find_grade(average):
                if average >= 90:
                    return "A"
                elif average >= 75:
                    return "B"
                elif average >= 60:
                    return "C"
                elif average >= 40:
                    return "D"
                else:
    return "F"
    def find_result(m1, m2, m3):
        if m1 >= 40 and m2 >= 40 and m3 >= 40:
            return "PASS"
            return "FAIL"
            roll = input("Enter roll number: ")
            name = input("Enter name: ")
            m1 = int(input("Enter marks in Subject 1: "))
            m2 = int(input("Enter marks in Subject 2: "))
            m3 = int(input("Enter marks in Subject 3: "))
            total = calculate_total(m1, m2, m3)
            average = calculate_average(total)
            grade = find_grade(average)
            result = find_result(m1, m2, m3)
            print("\n--- STUDENT RESULT ---")
            print("Roll Number:", roll)
            print("Name:", name)
            print("Subject 1:", m1)
            print("Subject 2:", m2)
            print("Subject 3:", m3)
            print("Total:", total)
            print("Average:", average)
            print("Grade:", grade)
            print("Result:", result)
```

Input and Output:

```
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\35.py ===
Enter roll number: 101
Enter name: ajay
Enter marks in Subject 1: 90
Enter marks in Subject 2: 91
Enter marks in Subject 3: 67

--- STUDENT RESULT ---
Roll Number: 101
Name: ajay
Subject 1: 90
Subject 2: 91
Subject 3: 67
Total: 248
Average: 82.66666666666667
Grade: B
Result: PASS
```

Result:

The program correctly calculates total, average, grade and result according to the given grading and pass criteria.

Question 36: ATM Simulation. Write a menu-driven program that verifies a PIN, allowing three attempts, and then permits

the customer to check the balance, withdraw cash and deposit cash. Write the functions check_pin(), is_valid_amount(), withdraw() and deposit(), where the last two return the updated balance.

Program:

```
PIN = "1234"
def check_pin():
    for attempt in range(3):
        entered = input("Enter PIN: ")
        if entered == PIN:
            return True
            print("Incorrect PIN.")
            return False
            def is_valid_amount(amount):
                return amount > 0
                def withdraw(balance, amount):
                    if not is_valid_amount(amount):
                        print("Invalid amount.")
                    elif amount > balance:
                        print("Insufficient balance.")
                    else:
                        balance -= amount
                        print("Withdrawal successful.")
                        return balance
def deposit(balance, amount):
    if not is_valid_amount(amount):
        print("Invalid amount.")
    else:
        balance += amount
        print("Deposit successful.")
        return balance
        if check_pin():
            balance = 5000.0
            while True:
                print("\n1. Check Balance")
                print("2. Withdraw")
                print("3. Deposit")
                print("4. Exit")
                choice = input("Enter choice: ")
                if choice == "1":
                    print("Balance =", balance)
                elif choice == "2":
                    amount = float(input("Enter amount to withdraw: "))
                    balance = withdraw(balance, amount)
                elif choice == "3":
                    amount = float(input("Enter amount to deposit: "))
                    balance = deposit(balance, amount)
                elif choice == "4":
                    print("Thank you.")
```

```
                    break
                else:
                    print("Invalid choice.")
            else:
                print("Card blocked after three incorrect attempts.")
```

Input and Output:

```
>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\36.py ====
Enter PIN: 1234

1. Check Balance
2. Withdraw
3. Deposit
4. Exit
Enter choice: 2
Enter amount to withdraw: 499
Withdrawal successful.

1. Check Balance
2. Withdraw
3. Deposit
4. Exit
Enter choice:
```

Result:

The program verifies the PIN for up to three attempts and correctly updates the balance after withdrawal and deposit.

Question 37: Simple Calculator. Write separate functions for addition, subtraction, multiplication and division, and a menu-

driven main program that calls the appropriate function. Division by zero must be handled inside the function.

Program:

```
def add(a, b):
    return a + b
    def subtract(a, b):
        return a - b
        def multiply(a, b):
            return a * b
            def divide(a, b):
                if b == 0:
return "Error: Division by zero is not allowed."
return a / b
while True:
    print("\n1. Addition")
    print("2. Subtraction")
    print("3. Multiplication")
    print("4. Division")
    print("5. Exit")
    choice = input("Enter choice: ")
    if choice == "5":
        break
```

```
        a = float(input("Enter first number: "))
        b = float(input("Enter second number: "))
        if choice == "1":
            print("Result =", add(a, b))
        elif choice == "2":
            print("Result =", subtract(a, b))
        elif choice == "3":
            print("Result =", multiply(a, b))
        elif choice == "4":
            print("Result =", divide(a, b))
        else:
            print("Invalid choice.")
```

Input and Output:

```
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\37.py =

1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter choice: 3
Enter first number: 23
Enter second number: 5
Result = 115.0

1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter choice:
```

Result:

The calculator performs all four operations and handles division by zero inside the division function.

Question 38: Electricity Bill. Write a function that returns the bill amount for the units consumed, using the slab rates Rs. 3

per unit for the first 100 units, Rs. 5 per unit for the next 200 units and Rs. 8 per unit thereafter.

Program:

```
def electricity_bill(units):
    if units <= 100:
        return units * 3
    elif units <= 300:
        return 100 * 3 + (units - 100) * 5
    else:
        return 100 * 3 + 200 * 5 + (units - 300) * 8
        units = int(input("Enter units consumed: "))
        print("Electricity bill = Rs.", electricity_bill(units))
```

Input and Output:

```
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\38.py =
Enter units consumed: 45
Electricity bill = Rs. 135
```

Result:

For 350 units, the bill is Rs. 300 for the first 100 units, Rs. 1000 for the next 200 units and Rs. 400 for the remaining 50 units

Question 39: Employee Salary. Write functions to compute the dearness allowance at 40% of the basic pay, the house rent

allowance at 20% of the basic pay, and the net salary. Display the complete salary slip.

Program:

```
    def calculate_da(basic):
        return basic * 0.40
        def calculate_hra(basic):
            return basic * 0.20
            def calculate_net_salary(basic):
    da = calculate_da(basic)
    hra = calculate_hra(basic)
    return basic + da + hra
    basic = float(input("Enter basic pay: "))
    da = calculate_da(basic)
    hra = calculate_hra(basic)
    net = calculate_net_salary(basic)
    print("\n--- SALARY SLIP ---")
    print("Basic Pay:", basic)
    print("DA (40%):", da)
    print("HRA (20%):", hra)
    print("Net Salary:", net)
```

Input and Output:

```
>>>
    ===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\39.py =
    Enter basic pay: 35000

    --- SALARY SLIP ---
    Basic Pay: 35000.0
    DA (40%): 14000.0
    HRA (20%): 7000.0
    Net Salary: 56000.0
```

Result:

The program correctly calculates DA at 40%, HRA at 20%, and net salary as basic pay plus both allowances.

Question 40: Library Management. Write a menu-driven program with the functions add_book(), display_books(),

search_book() and delete_book(), where the search function returns the position of the book and returns −1 when the book is absent.

Program:

```
    books = []
    def add_book():
        book = input("Enter book name: ")
```

```
        books.append(book)
        print("Book added.")
        def display_books():
            if not books:
                print("No books available.")
            else:
                for i in range(len(books)):
                    print(i, "-", books[i])
                    def search_book(name):
                        for i in range(len(books)):
                            if books[i].lower() == name.lower():
                                return i
                                return -1
                                def delete_book():
                                    name = input("Enter book name to delete: ")
                                    pos = search_book(name)
                                    if pos == -1:
                                        print("Book not found.")
                                    else:
                                        books.pop(pos)
                                        print("Book deleted.")
                                        while True:
                                            print("\n1. Add Book")
                                            print("2. Display Books")
                                            print("3. Search Book")
                                            print("4. Delete Book")
                                            print("5. Exit")
                                            choice = input("Enter choice: ")
                                            if choice == "1":
                                                add_book()
                                            elif choice == "2":
                                                display_books()
                                            elif choice == "3":
                                                name = input("Enter book name to search: ")
                                                print("Position =", search_book(name))
                                            elif choice == "4":

    delete_book()
    elif choice == "5":
        break
    else:
        print("Invalid choice.")
```

Input and Output:

```
>>>
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\40.py ===

1. Add Book
2. Display Books
3. Search Book
4. Delete Book
5. Exit
Enter choice: 2
No books available.

1. Add Book
2. Display Books
3. Search Book
4. Delete Book
5. Exit
Enter choice: 3
Enter book name to search: python
Position = -1

1. Add Book
2. Display Books
3. Search Book
4. Delete Book
5. Exit
Enter choice:
```

Result:

The library program correctly adds, displays, searches and deletes books. search_book() returns −1 when a book is absent.

Question 41: Number Analysis Menu. Write a menu-driven program offering the options factorial, prime check, sum of digits,

palindrome check and Armstrong check, by reusing the functions written in Section A. The menu must repeat until the user chooses to exit.

Program:

```
def factorial(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
        return result
        def is_prime(n):
            if n < 2:
                return False
                for i in range(2, int(n ** 0.5) + 1):
                    if n % i == 0:
                        return False
                        return True
                        def sum_digits(n):
                            n = abs(n)
                            total = 0
                            while n > 0:
                                total += n % 10
```

```
n //= 10
    return total
    def is_palindrome(n):
        s = str(n)
        return s == s[::-1]
        def is_armstrong(n):
            digits = str(n)
            power = len(digits)
            total = 0
            for d in digits:
                total += int(d) ** power
                return total == n
                while True:
                    print("\n1. Factorial")
                    print("2. Prime Check")
                    print("3. Sum of Digits")
                    print("4. Palindrome Check")
                    print("5. Armstrong Check")
                    print("6. Exit")
                    choice = input("Enter choice: ")
    if choice == "6":
        print("Program ended.")
        break
        n = int(input("Enter number: "))
        if choice == "1":
            print("Factorial =", factorial(n))
        elif choice == "2":
            print("Prime =", is_prime(n))
        elif choice == "3":
            print("Sum of digits =", sum_digits(n))
        elif choice == "4":
            print("Palindrome =", is_palindrome(n))
        elif choice == "5":
            print("Armstrong =", is_armstrong(n))
        else:
            print("Invalid choice.")
```

Input and Output:

```
===== RESTART: C:\Users\prana\OneDrive\Desktop\AV.SC.U4AIE25102\Lab 6\41.py =

1. Factorial
2. Prime Check
3. Sum of Digits
4. Palindrome Check
5. Armstrong Check
6. Exit
Enter choice: 2
Enter number: 2341
Prime = True

1. Factorial
2. Prime Check
3. Sum of Digits
4. Palindrome Check
5. Armstrong Check
6. Exit
Enter choice:
```

Result:

The menu repeatedly performs factorial, prime, digit-sum, palindrome and Armstrong checks and exits when the user selects Exit.

4. Observations

- Functions make programs modular, reusable and easier to understand.
- List-processing functions can be used to calculate averages, separate values, search, sort and remove duplicates.
- Variable-length arguments such as *subjects allow a function to accept any number of values.
- Menu-driven programs use repeated choices and separate functions to organize different operations.
- Functions can return updated values, such as the balance after withdrawal or deposit.
- Input validation and special cases, such as division by zero and insufficient balance, should be handled explicitly.

5. Overall Result

All the assigned programs from Questions 26 to 41 were implemented using Python functions and menu-driven logic. The sample inputs and outputs demonstrate the expected working of each program.